

# 数论-笔记

## 一、位运算

基本的位运算：共 6 种，分别为按位与、按位或、按位异或、按位取反、左移和右移。

Tips: 注意区分 逻辑与 ( $\wedge$ ) 和按位与 ( $\&$ )、逻辑或 ( $\vee$ ) 和按位或 ( $\mid$ ) 的区别。

### 异或:

$$a \wedge a = 0, a \wedge 0 = a$$

$$\text{交换 } a, b: a = a \wedge b, b = a \wedge b, a = a \wedge b$$

快速寻找数组中出现奇数次的数 (其他都为偶数)

$$a \text{ 是奇数 } a \wedge 1 = a - 1, \text{ 因为消除零 } // a \text{ 是偶数 } a \wedge 1 = a + 1$$

$$x \wedge (1 \ll n) \text{ 把 } x \text{ 的第 } i \text{ 位翻转}$$

$$a \text{ xor } (a \text{ xor } b) = (a \text{ xor } a) \text{ xor } b = 0 \text{ xor } b = b$$

### 取反:

是对一个数  $num$  进行的位运算，即单目运算。其对应的运算符为  $\sim$ 。

它的作用是把  $num$  的二进制补码中的 0 和 1 全部取反 (0 变为 1, 1 变为 0)。有符号整数的符号位在  $\sim$  运算中同样会取反。

**原码:** 最高位表示符号: 0 为正, 1 为负, 其余位表示数值的绝对值。

**补码:** 在二进制表示下, 正数和 0 的补码为其本身, 负数的补码是将其对应正数按位取反后+1。

**反码:** 正数: 反码 = 原码      负数: 符号位保持为 1, 其余位按位取反 (0 $\rightarrow$ 1, 1 $\rightarrow$ 0)。

**左移:**  $num \ll i$  表示将  $num$  的二进制表示向左移动  $i$  位所得的值。

**右移:**  $num \gg i$  表示将  $num$  的二进制表示向右移动  $i$  位所得的值。

$$\begin{aligned} 5 &= (00000101)_2 \\ \sim 5 &= (11111010)_2 = -6 \\ -5 \text{ 的补码} &= (11111011)_2 \\ \sim(-5) &= (00000100)_2 = 4 \end{aligned}$$

## 二、快速幂与快速乘整除性与约数

快速幂模板：

```
int fastpow(int a,int b)
{
    int ans = 1;
    while(b){
        if (b&1)  ans *= a;
        a *= a;
        b >>= 1;
    }
    return ans;
}
```

C++

快速乘模板：

```
int Mul(int a, int b){
    int ans = 0;
    while(b){
        if(b&1) ans += a;
        a+=a;
        b >>= 1;
    }
    return ans;
}
```

C++

简化阶乘求法:

C++

```
#include <stdio.h>
#include <math.h>

double funcos( double e, double x );

int main() {
    double e, x;
    scanf("%lf %lf", &e, &x);
    printf("cos(%.2f) = %.6f\n", x, funcos(e, x));
    return 0;
}

/* 你的代码将被嵌在这里 */

double funcos(double e, double x) {
    double sum = 1.0;        // 第0项: 1
    double term = 1.0;       // 当前项
    int n = 1;
    do{
        // 计算新项:  $(-1)^n * x^{(2n)} / (2n)!$ 
        term = term * (-1) * x * x / ((2*n-1) * (2*n));
        sum += term;
        n++;
    } while(fabs(term) >= e);
    return sum;
}
```

### 三、素数理论

素数的分布：开数组时空间除以10

埃及筛：

C++

```
const int MAXN = 1e6 + 5;
bool prim[MAXN];

void sieve(int n){
    fill(prim + 2, prim + n + 1, true);
    for (int i = 2; i * i <= n; i++){
        if (prim[i]){
            for (int j = i * i; j <= n; j += i)
                prim[j] = false;
        }
    }
}
```

线性筛（欧拉筛）：埃式筛（未去重）的升级版

原理：使用最小质因数去标记每个合数

```

const int N = 1e7;
vector<int> prim;
vector<int> p(N);
void primesa(){
    for(int i=2; i<N; ++i){
        if(p[i] == 0){
            p[i] = i;
            prim.push_back(i);
        }
        for(auto j : prim){
            if(i * j >= N) break;
            p[i*j] = j;
            if(i % j == 0) break;
            //意味着如果继续往下筛, `i * primes[j+1]` 的最小质因子将会是 `primes[j]` 而不是
            `primes[j+1]`, 这会导致重复标记。
        }
    }
}

```

### 区间筛:

场景: 求区间  $[L,R]$  内的素数, 其中  $R$  很大 (如 10121012), 但区间长度  $R-L+1$  较小 (如 106106)。

核心思想:  $[L,R]$  内的合数必有一个不超过  $\sqrt{R}$  的质因子。

1. 先预处理出  $1 \sim \sqrt{R}$  内的所有素数。
2. 用这些素数去筛掉区间  $[L,R]$  内的倍数。

```

typedef long long ll;
void segmented_sieve(ll L, ll R) {
    vector<bool> is_p(R - L + 1, true);
    if (L == 1) is_p[0] = false;

    for (ll i = 2; i * i <= R; i++) {
        if (isPrime(i)) { // 使用预处理的素数
            // 找到大于等于L的第一个i的倍数
            for (ll j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
                is_p[j - L] = false;
        }
    }
}

```

## 四、同余运算与剩余系

$\text{Gcd}(a,b) = \text{gcd}(b, a \% b)$

```
Int gcd(a,b){return !b?a:gcd(b,a%b);}
```

## 五、乘法逆元

定义: 如果  $a \cdot x \equiv 1 \pmod{m}$ , 我们就称  $x$  是  $a$  在模  $m$  下的乘法逆元, 记作  $a^{-1}$ 。

作用: 在模  $m$  意义下, 除以一个数等于乘以这个数的逆元。

$$a/b(mod m) = (a \cdot b^{-1})(mod m)$$

三个板子：

### 方法 A：费马小定理（最常用）

- 前提条件：m 必须是质数，且 a 不是 m 的倍数。
- 原理： $a^{m-2} \equiv a^{-1} (mod\ m)$ 。
- 实现：使用快速幂。

```
typedef long long ll;
ll fast_pow(ll a, ll b, ll m) {
    ll res = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}

// 求 a 在模 m 下的逆元
ll inv(ll a, ll m) {
    return fast_pow(a, m - 2, m);
}
```

C++

### 方法 B：扩展欧几里得算法 (exGCD)

- 前提条件：a 与 m 互质 ( $\gcd(a,m)=1$ )，m 不一定是质数。
- 原理：解方程  $ax+my=1$ ，求出的 x 即为逆元。

```
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
}

ll inv_exgcd(ll a, ll m) {
    ll x, y;
    ll d = exgcd(a, m, x, y);
    if (d != 1) return -1; // 无逆元
    return (x % m + m) % m; // 确保结果为正数
}
```

C++

### 方法 C：线性求逆元（求 1~n 所有数的逆元）

- 场景：当需要频繁用到 1~N 范围内的所有逆元（如组合数计算）。
- 时间复杂度： $O(N)O(N)$ 。

```

ll inv[MAXN];
void get_all_inv(int n, int m) {
    inv[1] = 1;
    for (int i = 2; i <= n; i++) {
        // 公式: inv[i] = (m - m/i) * inv[m%i] % m
        inv[i] = (long long)(m - m / i) * inv[m % i] % m;
    }
}

```

## 应用：带模除法与逆元

在计算  $ab \pmod m$  时，步骤如下：

1. 计算  $b$  的逆元  $b^{-1}$ 。
2. 结果为  $(a * b\_inv) \% m$ 。

```

int exp(int a, int b){
    int res = 1;
    a = a%mod;
    while(b){
        if(b%2 == 1) res=(a*res) % mod;
        a *= a;
        a = a%mod;
        b /= 2;
    }
    return res % mod;
int inv(int b){
    return exp(b,mod-2) % mod;
}
}

```

## 注意点：

- $aa$  本身可能是很大计算出来的结果，记得先对  $aa$  取模：  $(a \% m * b\_inv) \% m$ 。
- 只有当  $\gcd(b,m)=1$  时，逆元才存在。

## 应用：组合数取模板子： $Cnk(modp)$

```
const int MOD = 1e9 + 7;
const int MAXN = 100005;
ll fact[MAXN], invFact[MAXN];

ll fast_pow(ll a, ll b) { /* 见上文 */ }

// 预处理阶乘和阶乘的逆元
void init() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;

    // 阶乘逆元的推导：(n!)的逆元 = ((n+1)!)的逆元 * (n+1)
    invFact[MAXN - 1] = fast_pow(fact[MAXN - 1], MOD - 2);
    for (int i = MAXN - 2; i >= 0; i--) {
        invFact[i] = invFact[i + 1] * (i + 1) % MOD;
    }
}

ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    // 公式：n! / (r! * (n-r)!)
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}
```

## 六、欧拉函数与欧拉定理

## 七、离散对数

### 辗转相除法求gcd

```
long long llgcd(int a, int b) {
    // if(a>b) swap(a,b);
    if(b==0) return a;
    return llgcd(b, a%b);
}
```

### 通过gcd求最小公倍数

```
long long lcm(long long a, long long b) {
    if (a == 0 || b == 0) return 0;
    long long g = llgcd(a, b);
    return llabs((a / g) * b); //先除后乘，减少溢出风险_
}
```

### 求两个数的最大公共质因数

// 欧几里得算法

```
long long gcd_ll(long long a, long long b) {
    while (b){
        long long r = a % b;
        a = b;
        b = r;
    }
    return a >= 0 ? a : -a;
}

// n 的最大质因数 (n<=1 返回 n, 本题里 g=0/1 需要特判)
long long max_prime_factor(long long n) {
    if (n <= 1) return n;
    long long ans = -1;
    while (n % 2 == 0){
        ans = 2;
        n /= 2;
    }
    for (long long d = 3; d * d <= n; d += 2){
        while (n % d == 0){
            ans = d;
            n /= d;
        }
    }
    if (n > 1) ans = n;
    return ans;
}
```